Multi-label Classification với mô hình C2AE (Canonical Correlated AutoEncoder)

Nguyễn Mạnh Khiêm Nguyễn Phúc Thịnh Nguyễn Thế Hào Đinh Quang Vinh

I. Giới thiệu

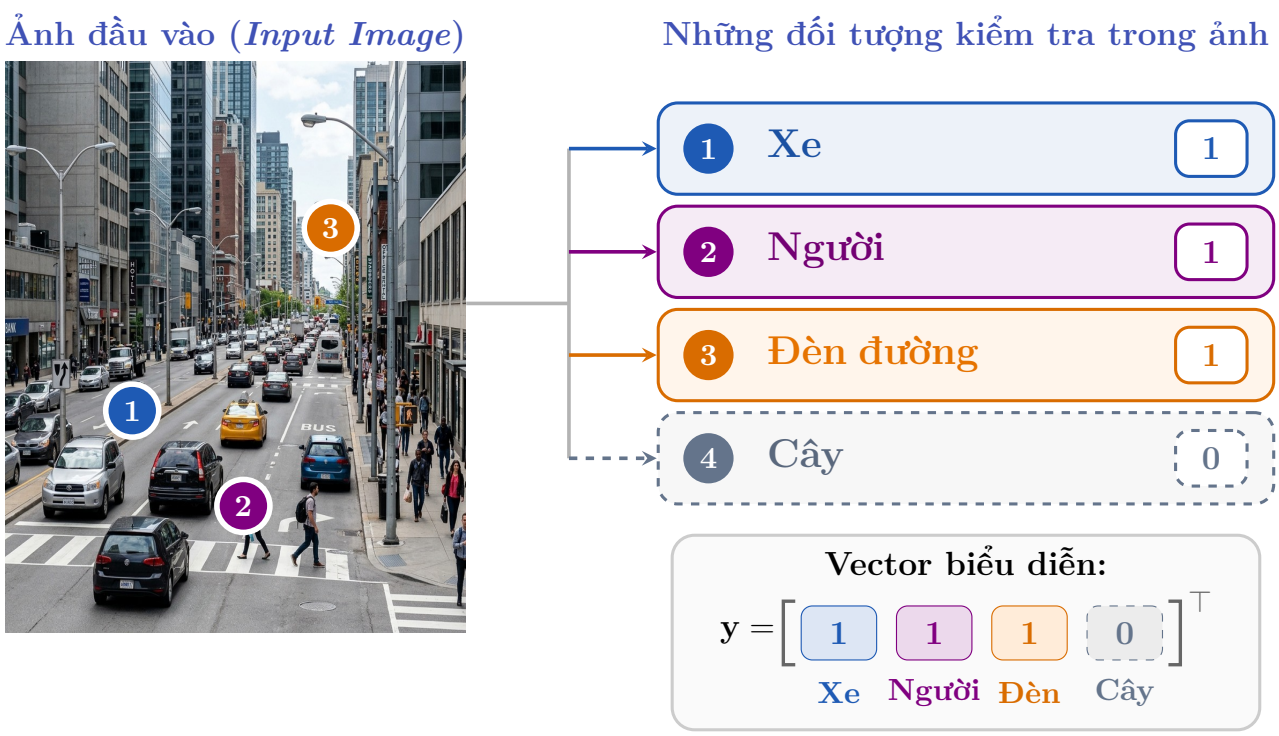
Trong lần khảo sát thực tế tại phòng điều hành an ninh ở một khu chung cư tại TP.HCM, nhóm tác giả nhận thấy nhân viên giám sát thường mất hàng giờ xem lại các đoạn băng camera một cách thủ công. Mỗi khi cần trích xuất sự việc theo yêu cầu của ban quản lý hoặc cư dân, việc tua đi tua lại hàng chục luồng video vừa tốn công sức, vừa rất dễ bỏ sót các thời điểm quan trọng.

Qua trao đổi, đội ngũ an ninh chia sẻ rằng nhu cầu tra cứu thực tế không đơn giản là tìm kiếm từng sự vật đơn lẻ, mà luôn đòi hỏi lọc nhanh theo tổ hợp nhiều điều kiện cùng lúc:

- Tìm kiếm hình ảnh có **phương tiện** di chuyển cùng lúc với **người giao hàng** tại sảnh chung cư vào giờ cao điểm.
- Rà soát khu vực khuôn viên xuất hiện **người đi bộ** dắt theo thú cưng vào khung giờ muộn.
- Phát hiện các đoạn đường nội bộ có **xe cộ** lưu thông trong khi đang có **vật cản** hoặc **người đi bộ qua đường**.

Nếu hệ thống Camera AI truyền thống chỉ phân loại đơn nhãn (ví dụ gán nhãn duy nhất là "Xe"), nó sẽ bỏ qua hoàn toàn các đối tượng khác cùng hiện diện. Nhu cầu thực tế đó dẫn chúng ta đến bài toán nhận diện đồng thời nhiều đối tượng trong một khung hình, hay chính là bài toán **Phân loại đa nhãn** (Multi-label Classification).

Như minh họa ở Hình 1, bức ảnh ghi nhận đồng thời ba đối tượng (Xe, Người, Đèn đường) nhưng không có Cây, tương ứng với vector nhãn $\mathbf{y} = [1, 1, 1, 0]^\top$.



Hình 1: Minh họa bài toán phân loại đa nhãn trên ảnh camera giao thông với vector nhãn $\mathbf{y} = [1, 1, 1, 0]^\top$.

Hai cách tiếp cận cơ bản cho bài toán phân loại đa nhãn là:

- **Binary Relevance:** Dùng L bộ phân loại nhị phân độc lập, đơn giản nhưng không mô hình hóa tương quan giữa các nhãn.
- **Label Powerset:** Coi mỗi tổ hợp nhãn đã thấy là một lớp. Với tối đa 2^L tổ hợp, dữ liệu bị chia nhỏ và các lớp hiếm không đủ mẫu để học.

Hạn chế của các phương pháp này sẽ được phân tích chi tiết ở Mục II.

Từ nghịch lý giữa việc đơn giản hóa mô hình và việc nắm bắt tương quan nhãn, hai câu hỏi lớn được đặt ra:

1 Làm sao tránh bùng nổ tổ hợp nhãn?

Làm sao dự đoán đồng thời L nhãn trong **một mô hình**, mà không phải coi mỗi tổ hợp nhãn là một lớp riêng?

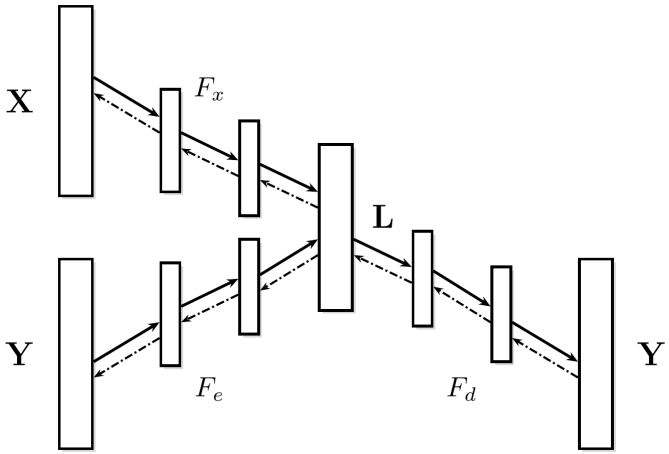
2 Làm sao để học tương quan giữa các nhãn?

Làm sao để mô hình **tự học mối liên kết** (*correlation*) giữa các nhãn thay vì phán đoán độc lập?

Đề xuất mang tên Canonical Correlated AutoEncoder (C2AE)

Vào năm 2017, Chih-Kuan Yeh cùng các cộng sự tại Academia Sinica và Đại học Quốc gia Đài Loan đã công bố mô hình mang tên **C2AE** (*Canonical Correlated AutoEncoder*) [1]. Hướng

tiếp cận này mang lại một ý tưởng nền tảng giúp giải quyết đồng thời cả hai câu hỏi được đặt ra ở trên, vừa giữ kiến trúc suy luận gọn gàng, vừa nắm bắt sâu sắc mối tương quan giữa các nhãn thông qua việc kết hợp không gian ẩn của nhãn và đặc trưng dữ liệu. C2AE học đồng thời bộ mã hóa đặc trưng (F_x), bộ mã hóa nhãn (F_e) và bộ giải mã nhãn (F_d) trong một mô hình thống nhất.



Hình 2: Kiến trúc tổng quan của mô hình C2AE trích từ Yeh và cộng sự [1].

Mục tiêu của bài viết này là giúp người đọc hiểu được cơ chế của C2AE, bao gồm động lực thiết kế, cơ sở toán học, kiến trúc mô hình, ví dụ tính tay cụ thể, cùng việc cài đặt bằng PyTorch và đánh giá thực nghiệm.

Mục lục

I. Giới thiệu **1**

II. Bản chất bài toán phân loại đa nhãn **8**

1. Từ phân loại nhị phân đến phân loại đa nhãn 8
2. Các phương pháp tiếp cận truyền thống 9

III. Ý tưởng và nguyên lý thiết kế mô hình C2AE **11**

1. Ôn tập kiến trúc AutoEncoder 11
2. Tư duy hình thành kiến trúc C2AE 12
3. Bức tranh toàn cảnh giữa pha Huấn luyện và Suy luận 15

IV. Ví dụ tính tay từng bước Forward **16**

1. Thiết lập ví dụ 16
2. Khởi tạo trọng số các mạng thành phần 16
3. Bước 1: Tính Forward qua Feature Encoder (F_x) 16
4. Bước 2: Tính Forward qua Label Encoder (F_e) 17
5. Bước 3: Tính Forward qua Label Decoder (F_d) 18

V. Hàm mất mát xếp hạng nhãn $\Gamma(S, Y)$ **19**

VI. Căn chỉnh không gian tiềm ẩn $\Phi(A, B)$ **21**

1. Kéo hai biểu diễn lại gần nhau 21
2. Giữ các chiều biểu diễn không suy biến 21
3. Ghép hai thành phần thành $\Phi(A, B)$ 22
4. Hàm mục tiêu chung và cách cập nhật mô hình 23

VII. Suy luận và chọn ngưỡng dự đoán **23**

1. Dùng nhánh ảnh thay cho nhánh nhãn 23
2. Đổi điểm số thành nhãn 24
3. Chọn ngưỡng trên tập validation 25

VIII. Các độ đo chuyên dụng cho bài toán đa nhãn **25**

1. Hamming Loss: Sai bao nhiêu ô nhãn? 26
2. Subset Accuracy: Bao nhiêu mẫu đúng hoàn toàn? 26
3. Precision, Recall và F1 của từng nhãn 26

4. Micro-F1: Gộp số đếm rồi tính F1 28
5. Macro-F1: Tính F1 rồi lấy trung bình các nhãn 28

IX. Triển khai thực tế mô hình C2AE với PyTorch 28

1. Thiết lập bài toán và cấu trúc dữ liệu mô phỏng 28
2. Bước 1: Chuẩn hóa đặc trưng đầu vào 29
3. Bước 2: Xây dựng kiến trúc mô hình C2AE 29
4. Bước 3: Cài đặt hai hàm mất mát tùy biến 30
5. Bước 4: Huấn luyện và Cân chỉnh ngưỡng tối ưu 31
6. Bước 5: Đánh giá độc lập trên tập Test 33

X. Tổng kết 33

Tài liệu tham khảo 34

Để thuận tiện trong quá trình đọc và nghiên cứu, dưới đây là bảng thuật ngữ và bảng ký hiệu toán học quy chuẩn được sử dụng xuyên suốt toàn bài.

Bảng tra cứu thuật ngữ chuyên ngành

Thuật ngữ	Định nghĩa & Ý nghĩa trong bài
Multi-class Classification	Phân loại nhiều lớp, trong đó mỗi mẫu dữ liệu chỉ thuộc về duy nhất một lớp loại trừ lẫn nhau (vector one-hot).
Multi-label Classification	Phân loại đa nhãn, trong đó mỗi mẫu dữ liệu có thể đồng thời thuộc về nhiều lớp khác nhau (vector multi-hot).
Label Correlation	Tương quan nhãn, biểu thị xu hướng các nhãn đồng xuất hiện (co-occurrence) hoặc loại trừ lẫn nhau trong thực tế.
Shared Latent Space	Không gian tiềm ẩn dùng chung, đóng vai trò là không gian vector trung gian r chiều nơi cả đặc trưng X và nhãn Y cùng được ánh xạ về.
AutoEncoder (AE)	Mạng tự mã hóa gồm Encoder tạo biểu diễn tiềm ẩn và Decoder tái tạo đầu vào. Dạng undercomplete dùng biểu diễn có số chiều nhỏ hơn đầu vào.
Canonical Correlation Analysis (CCA)	Một kỹ thuật thống kê giúp phân tích tương quan, tìm các phép chiếu tuyến tính sao cho các biến chiếu tương ứng của hai tập biến ngẫu nhiên đạt tương quan cực đại.
Representation Collapse	Suy biến biểu diễn, hiện tượng mạng tối ưu hội tụ về nghiệm tầm thường (ví dụ tất cả vector đều bằng 0 hoặc trùng vào một điểm duy nhất).
Gram Matrix	Ma trận Gram ($A^\top A$), ma trận tích vô hướng đo lường độ lớn và tính trực giao giữa các chiều latent trên batch.
Label Ranking Loss	Hàm mất mát xếp hạng nhãn, phạt các cặp nhãn theo chênh lệch điểm số, thúc đẩy điểm của nhãn dương cao hơn nhãn âm.
Threshold Calibration	Cân chỉnh ngưỡng, quá trình quét tìm giá trị ngưỡng tối ưu trên tập Validation để phân định nhãn có mặt hay vắng mặt.
Hamming Loss	Tỷ lệ dự đoán sai trên từng ô nhãn riêng lẻ (đo độ chính xác cục bộ).
Subset Accuracy	Tỷ lệ mẫu có toàn bộ 100% các nhãn dự đoán đúng hoàn toàn (đo độ chính xác toàn cục, khắt khe).

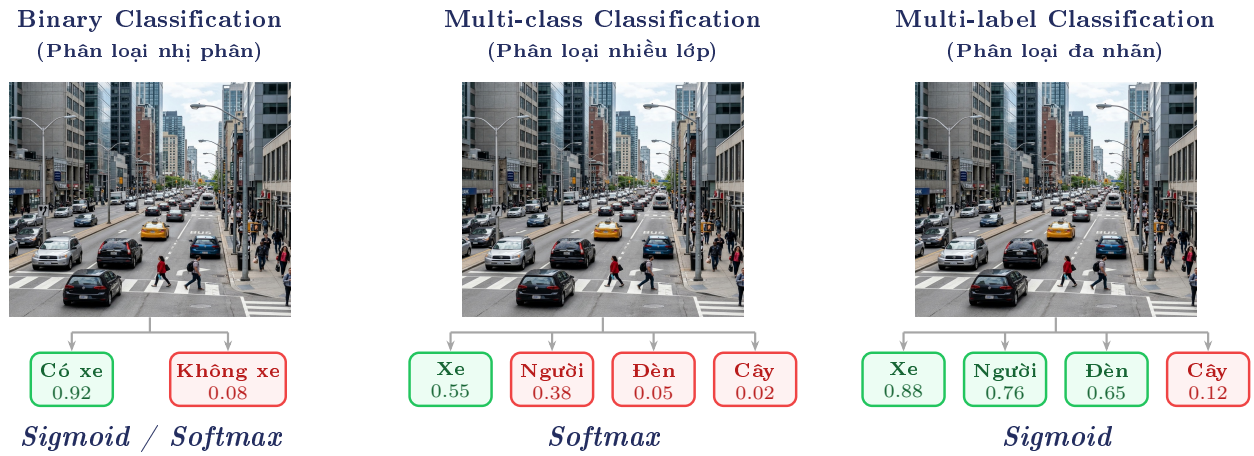
Bảng ký hiệu toán học quy chuẩn

Ký hiệu	Ý nghĩa toán học
N	Kích thước batch (số lượng mẫu dữ liệu xử lý đồng thời).
d	Số chiều của vector đặc trưng đầu vào.
L	Tổng số nhãn cần dự đoán.
r	Số chiều của không gian tiềm ẩn dùng chung (chọn trên tập validation), trong đó giá trị r nhỏ giúp nén biểu diễn cô đọng.
X	Ma trận đặc trưng của cả batch, mỗi hàng là một vector $x_i^\top$.
Y	Ma trận nhãn nhị phân thật của batch, mỗi hàng là một vector multi-hot $y_i^\top$.
$F_x(\cdot)$	Mạng Feature Encoder, ánh xạ từ không gian đặc trưng vào không gian ẩn.
$F_e(\cdot)$	Mạng Label Encoder, ánh xạ từ không gian nhãn vào không gian ẩn.
$F_d(\cdot)$	Mạng Label Decoder, giải mã từ không gian ẩn ra điểm số nhãn.
$A = F_x(X)$	Ma trận biểu diễn latent sinh ra từ nhánh đặc trưng.
$B = F_e(Y)$	Ma trận biểu diễn latent sinh ra từ nhánh nhãn.
$S^{\text{train}} = F_d(B)$	Ma trận điểm số nhãn giải mã từ B trong quá trình huấn luyện ($N \times L$).
$S^{\text{infer}} = F_d(A)$	Ma trận điểm số nhãn giải mã từ A trong quá trình suy luận test ($N \times L$).
$P_i,\ Q_i$	Tập chỉ số của các nhãn dương ($y_{ij} = 1$) và nhãn âm ($y_{ij} = 0$) của mẫu i.
$\Phi(A, B)$	Hàm mất mát căn chỉnh không gian ẩn (Latent Alignment Loss) kèm phạt Gram.
$\Gamma(S, Y)$	Hàm mất mát xếp hạng nhãn (Label Ranking Loss).
$\mathcal{J} = \Phi + \alpha\Gamma$	Hàm mục tiêu tối ưu kết hợp toàn diện của C2AE.
$\lambda,\ \alpha$	Siêu tham số điều tiết hàm phạt Gram và tỷ trọng mất mát xếp hạng.
I_r	Ma trận đơn vị kích thước $r \times r$.
$\|M\|_{\text{F}}^2$	Chuẩn Frobenius bình phương của ma trận M, tính bằng tổng bình phương tất cả phần tử trong M.
t	Ngưỡng quyết định nhãn ($s_{ij} \geq t \implies \hat{y}_{ij} = 1$).

II. Bản chất bài toán phân loại đa nhãn

1. Từ phân loại nhị phân đến phân loại đa nhãn

Trong học máy giám sát (supervised learning), sự khác biệt giữa các bài toán phân loại bắt nguồn từ cách mô hình ràng buộc mối quan hệ giữa các nhãn đầu ra (Hình 3):

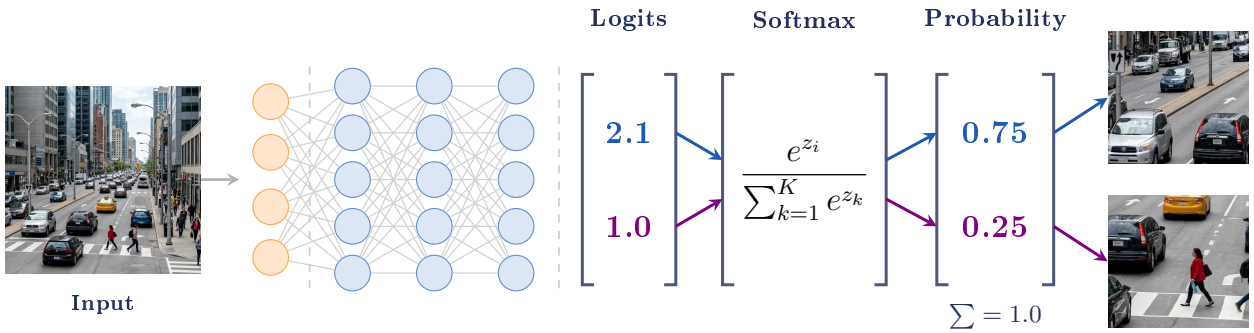


Hình 3: So sánh không gian nhãn giữa phân loại nhị phân, đa lớp và đa nhãn.

- **Phân loại nhị phân (Binary Classification)**: Dự đoán một thuộc tính nhị phân duy nhất $y \in \{0, 1\}$ (ví dụ: ảnh có xe hay không) qua một hàm Sigmoid với xác suất $p \in [0, 1]$.
- **Phân loại nhiều lớp (Multi-class Classification)**: Mỗi mẫu thuộc về duy nhất một trong C lớp ứng viên ($\sum_{c=1}^{C} y_c = 1$). Mô hình sử dụng hàm kích hoạt **Softmax**:

$$P(y = c \mid x) = \frac{e^{z_c}}{\sum_{k=1}^{C} e^{z_k}}. \tag{1}$$

  Do tổng xác suất bị cố định bằng 1, các lớp cạnh tranh lẫn nhau. Khi xác suất của lớp chiếm ưu thế (Xe, 0.75) tăng lên, nó trực tiếp kéo giảm xác suất của đối tượng cùng hiện diện (Người đi bộ) xuống chỉ còn 0.25, khiến đối tượng này bị loại bỏ khi đưa ra quyết định phân loại (Hình 4).



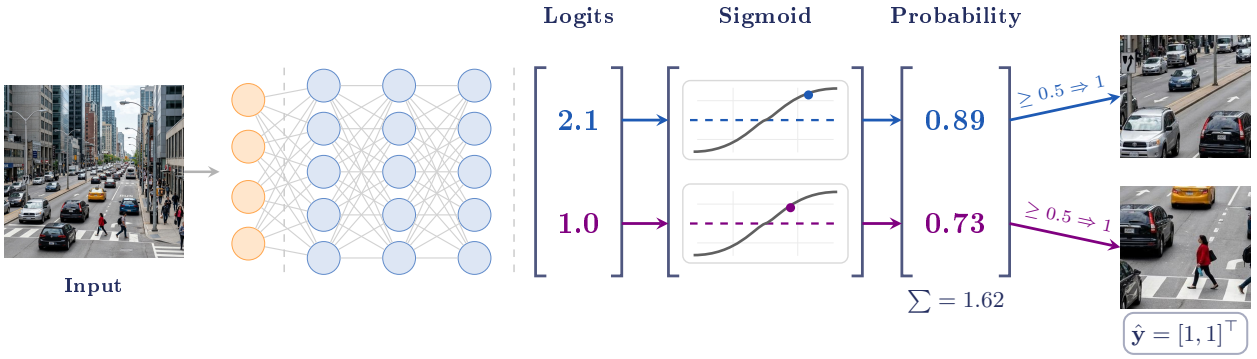
Hình 4: Cơ chế cạnh tranh của Softmax: lớp chiếm ưu thế kéo giảm xác suất của các đối tượng cùng hiện diện.

- **Phân loại đa nhãn (Multi-label Classification)**: Mỗi mẫu có thể đồng thời chứa bất kỳ tập con nào của không gian L nhãn $\mathcal{L} = \{\ell_1, \ldots, \ell_L\}$, biểu diễn bằng vector multi-hot $y = [y_1, \ldots, y_L]^\top \in \{0,1\}^L$ (với $y_j = 1$ nếu nhãn xuất hiện và 0 nếu vắng mặt).

  Để loại bỏ sự cạnh tranh triệt tiêu của Softmax, cách thông thường là chuyển sang sử dụng các hàm kích hoạt **Sigmoid** độc lập cho từng nơ-ron đầu ra:

$$P(y_j = 1 \mid x) = \sigma(z_j) = \frac{1}{1 + e^{-z_j}}, \quad \forall j \in \{1, \ldots, L\}. \tag{2}$$

  Mỗi đầu ra Sigmoid chỉ phụ thuộc vào logit tương ứng, nên các đầu ra không phải chia sẻ tổng xác suất bằng 1. Trong ví dụ ở Hình 5, cả Xe (0.89) và Người đi bộ (0.73) đều vượt ngưỡng $\tau = 0.5$, cho vector dự đoán $\hat{\mathbf{y}} = [1, 1]^\top$.

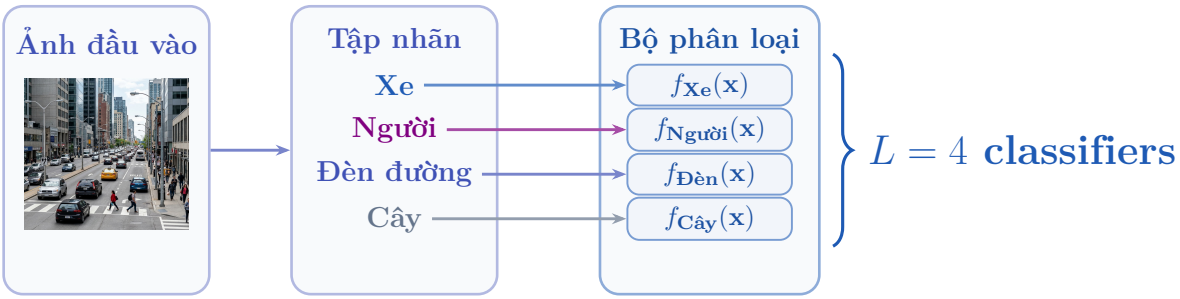


Hình 5: Cơ chế kích hoạt độc lập qua hàm Sigmoid với ngưỡng $\tau = 0.5$ trong phân loại đa nhãn.

2. Các phương pháp tiếp cận truyền thống

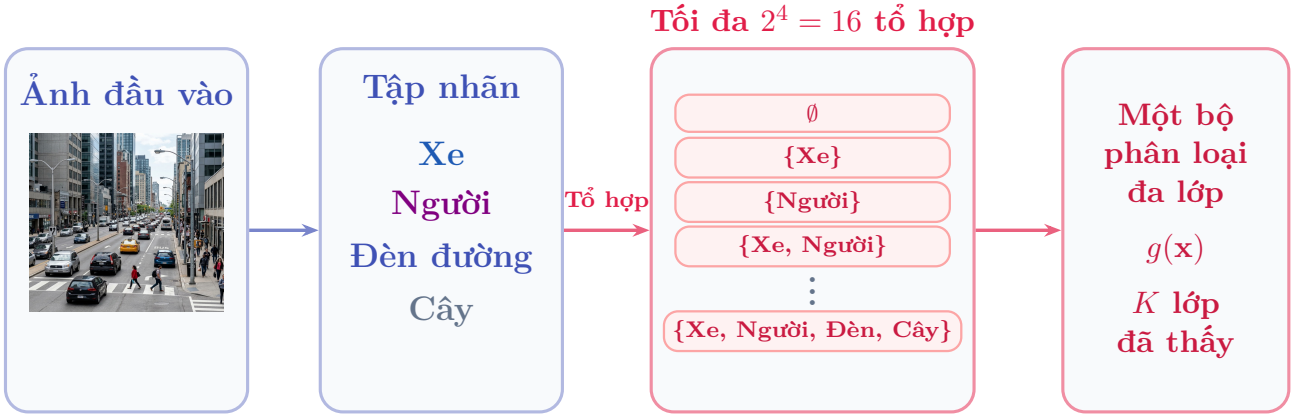
Để giải quyết bài toán phân loại đa nhãn, hai phương pháp chuyển đổi bài toán cơ bản là Binary Relevance và Label Powerset [2]:

- **Binary Relevance (BR)**: Huấn luyện L bộ phân loại nhị phân riêng biệt và độc lập hoàn toàn với nhau (Hình 6). Mặc dù đơn giản và có thể huấn luyện song song nhanh chóng, BR hoàn toàn bỏ qua mối tương quan giữa các nhãn (ví dụ: sự đồng xuất hiện giữa *Xe* và *Người*). Kết quả là mô hình dự đoán từng nhãn một cách cô lập, dễ dẫn đến những kết quả thiếu tính nhất quán tổng thể.



Hình 6: Phương pháp Binary Relevance với L bộ phân loại độc lập.

- **Label Powerset (LP)**: Xem mỗi tổ hợp nhãn đồng xuất hiện như một lớp duy nhất trong bài toán phân loại nhiều lớp. Hướng đi này biểu diễn trực tiếp các tổ hợp nhãn đã quan sát, nhưng gặp phải bài toán **bùng nổ không gian trạng thái** như minh họa ở Hình 7.



Hình 7: Label Powerset dùng một bộ phân loại đa lớp trên K tổ hợp đã thấy khi huấn luyện. Hình minh họa các tổ hợp khả dĩ (với $L = 4$ có tối đa 16 tổ hợp).

Với $L = 4$, bài toán chỉ có tối đa $2^4 = 16$ tổ hợp. Tuy nhiên khi $L = 20$, số tổ hợp khả dĩ bùng nổ lên tới hơn một triệu ($2^{20} = 1.048.576$). Việc có quá nhiều lớp khiến dữ liệu bị chia nhỏ, dẫn đến nhiều tổ hợp hiếm chỉ có 1–2 mẫu và không đủ để mô hình học. Ngoài ra khi suy luận, LP chỉ có thể chọn trong các tổ hợp đã thấy khi huấn luyện, hoàn toàn không thể dự đoán được một tổ hợp nhãn mới.

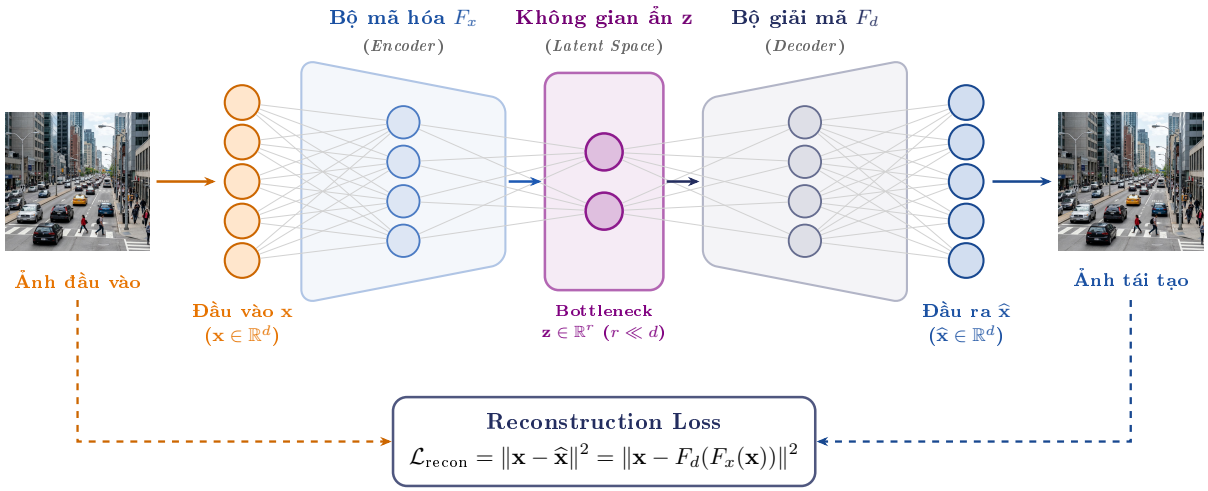
Chính những giới hạn đó đã đặt ra bài toán: **Làm thế nào để vừa nắm bắt sâu sắc tương quan nhãn, vừa tránh được sự bùng nổ tổ hợp số lớp?**

III. Ý tưởng và nguyên lý thiết kế mô hình C2AE

Mô hình **C2AE** là viết tắt của **C**anonical **C**orrelated **A**uto**E**ncoder (Mạng tự mã hóa tương quan chính tắc) [1], kết hợp giữa Phân tích tương quan chính tắc (Deep Canonical Correlation Analysis, DCCA) và Mạng tự mã hóa (AutoEncoder). Do mô hình được xây dựng dựa trên nền tảng AutoEncoder, trước hết chúng ta cần ôn tập lại các ý chính của kiến trúc này.

1. Ôn tập kiến trúc AutoEncoder

AutoEncoder học biểu diễn bằng cách tái tạo chính dữ liệu đầu vào, không cần nhãn bên ngoài (Unsupervised). Ở phần này, chúng ta sẽ xét mạng AutoEncoder có biểu diễn tiềm ẩn ít chiều hơn dữ liệu đầu vào, như minh họa ở Hình 8.



Hình 8: Kiến trúc AutoEncoder cơ bản với hàm mất mát tái tạo $\mathcal{L}_{\text{recon}}$.

Dựa trên sơ đồ ở Hình 8, mạng vận hành qua ba khối thành phần chính:

- **Bộ mã hóa (Encoder)** F_x: Nén dữ liệu đầu vào $\mathbf{x} \in \mathbb{R}^d$ qua các tầng nơ-ron thu hẹp dần để tạo thành biểu diễn ẩn $\mathbf{z} \in \mathbb{R}^r$ ($r \ll d$):

$$\mathbf{z} = F_x(\mathbf{x}). \tag{3}$$

- **Không gian ẩn Bottleneck (Bottleneck Latent Space)**: Tầng ẩn trung tâm có số chiều r nhỏ hơn đáng kể so với không gian gốc d. Nút thắt giới hạn lượng thông tin truyền sang decoder. Thông qua hàm mất mát tái tạo, encoder học cách giữ lại thông tin cần thiết để decoder khôi phục dữ liệu đầu vào.

- **Bộ giải mã (Decoder)** F_d: Mở rộng biểu diễn ẩn $\mathbf{z}$ qua các tầng nơ-ron để khôi phục lại dữ liệu ban đầu:

$$\hat{\mathbf{x}} = F_d(\mathbf{z}) \approx \mathbf{x}. \tag{4}$$

Quá trình huấn luyện diễn ra thông qua việc cực tiểu hóa hàm mất mát tái tạo (Reconstruction Loss):

$$\mathcal{L}_{\text{recon}} = \|\mathbf{x} - \hat{\mathbf{x}}\|^2 = \|\mathbf{x} - F_d(F_x(\mathbf{x}))\|^2. \tag{5}$$

Thông tin cần nhớ

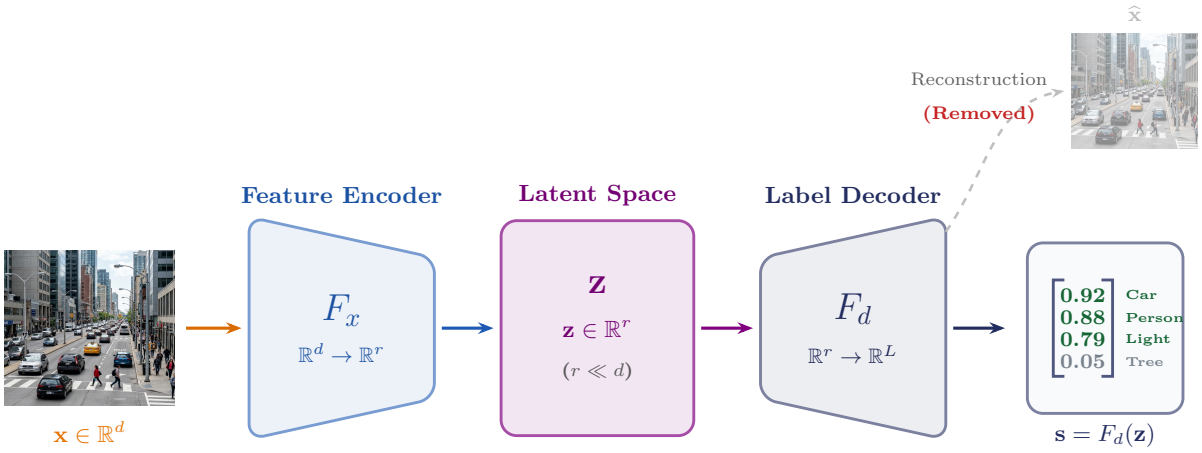
Mục đích của AutoEncoder không phải là tái tạo lại ảnh, mà là **học cách chắt lọc các đặc trưng cốt lõi vào không gian ẩn**. Vì phải đi qua một bottleneck có số chiều rất nhỏ, bộ mã hóa (Encoder) buộc phải giữ lại những thông tin quan trọng nhất để bộ giải mã (Decoder) có thể khôi phục lại dữ liệu ban đầu. Chính ý tưởng nén thông tin vào không gian ẩn này là nền tảng để C2AE học được mối liên kết giữa đặc trưng ảnh và nhãn.

2. Tư duy hình thành kiến trúc C2AE

Từ kiến trúc AutoEncoder cơ bản đã ôn tập ở trên, ý tưởng của C2AE được triển khai như sau:

Bước 1: Thay đầu ra từ tái tạo ảnh thành dự đoán nhãn y

Trong mô hình AutoEncoder truyền thống, mạng nhận ảnh $\mathbf{x}$ và cố gắng khôi phục lại chính bức ảnh đó ($\hat{\mathbf{x}} \approx \mathbf{x}$). Tuy nhiên, mục tiêu của bài toán phân loại đa nhãn không phải là vẽ lại ảnh, mà là nhận diện những nhãn nào đang xuất hiện. Vì vậy, bước chuyển đổi trực tiếp nhất là **thay thế đầu ra tái tạo ảnh bằng điểm số cho vector nhãn y**, như minh họa ở Hình 9.



Hình 9: Bước 1: Chuyển đổi đầu ra từ tái tạo ảnh sang dự đoán điểm số nhãn $\mathbf{s} = F_d(F_x(\mathbf{x}))$.

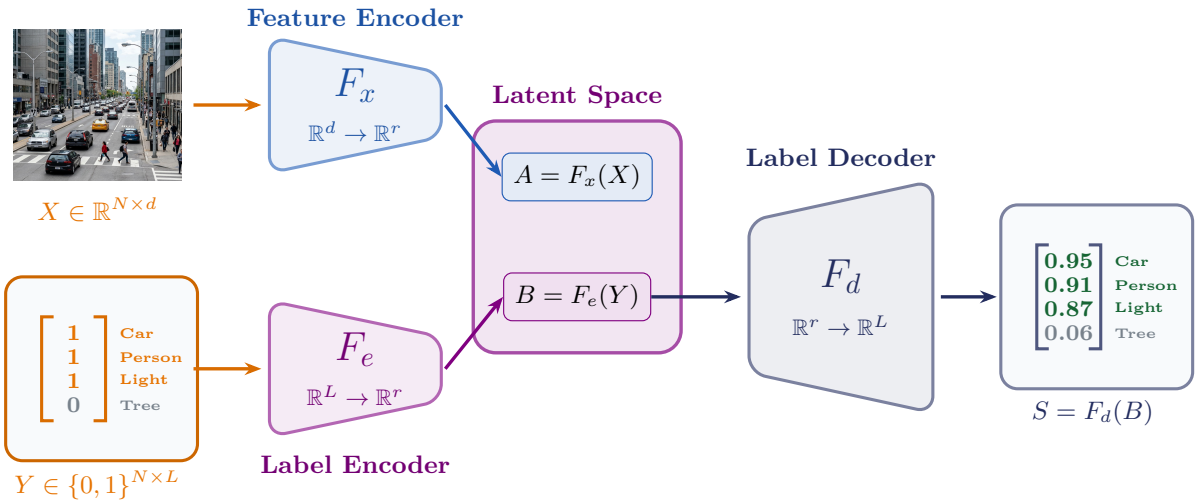
Tại đây, bộ mã hóa F_x nén đặc trưng ảnh thành vector ẩn $\mathbf{z}$, và bộ giải mã F_d ánh xạ từ $\mathbf{z}$ ra điểm số dự đoán cho từng nhãn.

Hạn chế: Ở bước này, nhãn thật được dùng làm mục tiêu giám sát cho đầu ra dự đoán. Cấu trúc của các tổ hợp nhãn chưa được học thành một biểu diễn riêng để hướng dẫn nhánh ảnh.

C2AE khai thác thêm thông tin này bằng cách học biểu diễn nhãn và căn chỉnh nó với biểu diễn ảnh.

Bước 2: Thêm nhánh đưa ma trận nhãn Y vào không gian ẩn (Latent Space)

Để khai thác cấu trúc nhãn, C2AE bổ sung bộ mã hóa nhãn F_e nhận ma trận $Y \in \{0,1\}^{N\times L}$ làm đầu vào. Nhánh này hoạt động song song với nhánh mã hóa đặc trưng ảnh X, như minh họa ở Hình 10. Điều này biến C2AE từ bài toán Unsupervised thành Supervised Learning.



Hình 10: Bước 2: Bổ sung nhánh mã hóa nhãn $F_e(Y)$ hội tụ vào không gian ẩn chung.

Việc đưa trực tiếp Y qua bộ mã hóa F_e giúp nén thông tin nhãn thành một biểu diễn ẩn có số chiều thấp hơn. Quá trình này chủ động cấu trúc hóa quy luật đồng xuất hiện của các nhãn trong không gian ẩn, từ đó học được tương quan nhãn trong cùng một biểu diễn dùng chung mà không phải tạo riêng một lớp phân loại cho từng tổ hợp như Label Powerset.

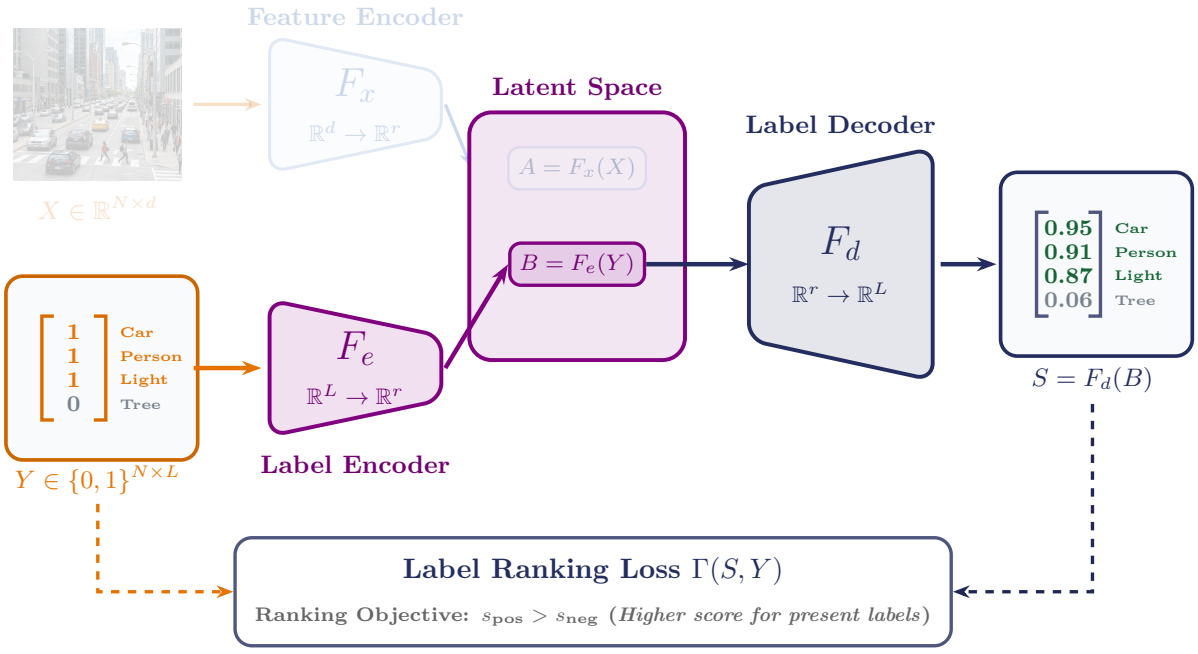
Thiết kế hàm mất mát tổng thể (Joint Loss)

Để toàn bộ hệ thống hoạt động thống nhất, C2AE kết hợp hai nhiệm vụ vào một hàm mất mát chung:

$$\min_{\theta_x,\theta_e,\theta_d} \mathcal{J} = \alpha\,\Gamma(S,Y) + \Phi(A,B), \tag{6}$$

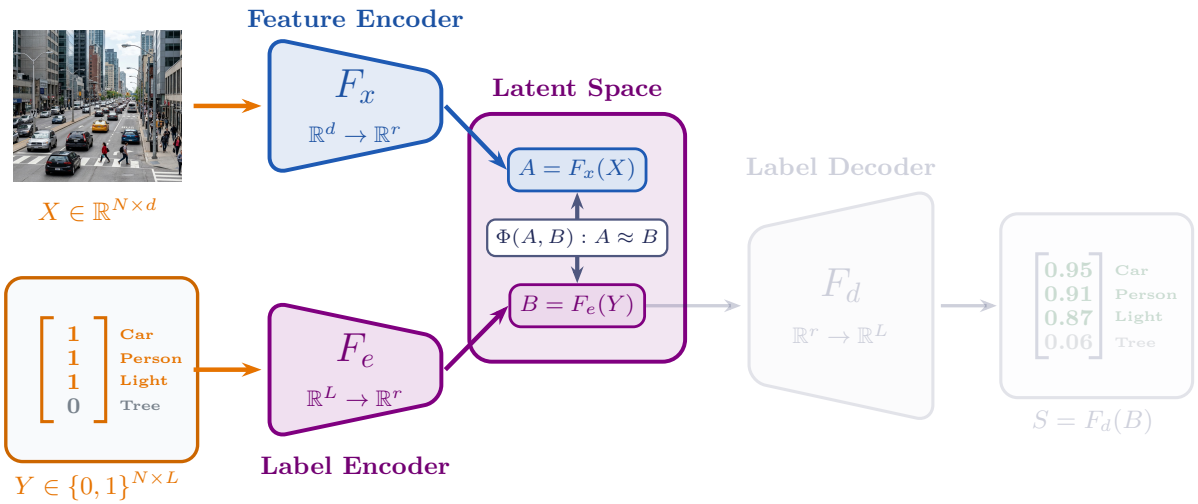
trong đó $A = F_x(X)$, $B = F_e(Y)$, $S = F_d(B)$, còn $\theta_x, \theta_e, \theta_d$ lần lượt là tham số của ba mạng. Trọng số $\alpha > 0$ cân bằng hai mục tiêu dưới đây.

1. Xếp hạng nhãn Γ (Label Ranking): So sánh điểm số $S = F_d(B)$ với nhãn thật Y, thúc đẩy nhãn dương cao điểm hơn nhãn âm. Loss được tính trên chu trình $Y \to F_e \to B \to F_d \to S$ và cập nhật F_e, F_d, như ở Hình 11. Phần V. trình bày cách tính từng cặp nhãn.



Hình 11: Xếp hạng nhãn Γ trên chu trình AutoEncoder nhãn.

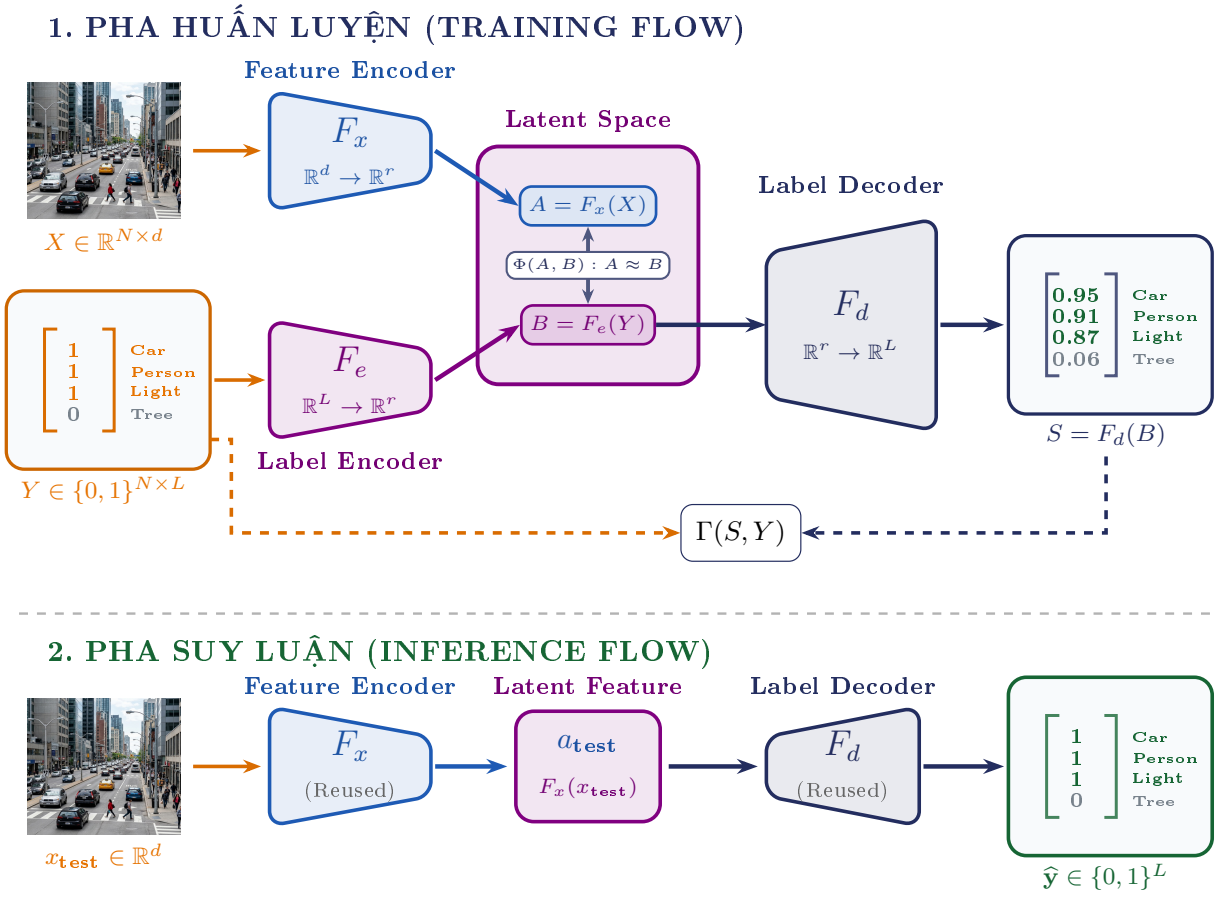
2. Đồng bộ không gian ẩn Φ (Latent Alignment): Kéo biểu diễn ảnh A gần biểu diễn nhãn B để decoder dùng được A khi suy luận. Loss cập nhật F_x, F_e và kèm phạt Gram để chống suy biến, như ở Hình 12. Phần VI. giải thích và tính tay hai thành phần này.



Hình 12: Đồng bộ không gian ẩn Φ giữa nhánh ảnh và nhánh nhãn.

3. Bức tranh toàn cảnh giữa pha Huấn luyện và Suy luận

Hình 13 minh họa chi tiết sự khác biệt giữa hai luồng dữ liệu trong mô hình C2AE:



Hình 13: Bức tranh toàn cảnh C2AE trong pha Huấn luyện (tối ưu Φ và Γ) và pha Suy luận ($F_x \to F_d$).

Trong pha huấn luyện, X và Y được mã hóa thành A và B. Tiếp đó, bộ giải mã (decoder) nhận B để tính điểm số S và mất mát xếp hạng $\Gamma(S, Y)$. Đồng thời, $\Phi(A, B)$ căn chỉnh hai biểu diễn và chống suy biến. Khi suy luận, mô hình chỉ cần đặc trưng đầu vào, thực hiện một lượt truyền qua F_x rồi F_d, sau đó phân ngưỡng điểm số để tạo vector nhãn nhị phân $\hat{\mathbf{y}}$ (cơ chế xác định ngưỡng tối ưu được trình bày chi tiết ở Mục VII.).

IV. Ví dụ tính tay từng bước Forward

Để hiểu rõ cơ chế hoạt động của C2AE, ta xét một ví dụ tính tay với các ma trận nhỏ. Ta sẽ lần lượt tính biểu diễn từ nhánh đặc trưng, biểu diễn từ nhánh nhãn và điểm số đầu ra của bộ giải mã, làm cơ sở để tính các hàm mất mát ở phần tiếp theo.

1. Thiết lập ví dụ

Xét ví dụ gồm hai mẫu ảnh ($n = 2$) với không gian bốn nhãn ($L = 4$) theo thứ tự: Xe, Người, Đèn, Cây. Mỗi ảnh có hai đặc trưng đầu vào ($d = 2$), đồng thời không gian tiềm ẩn cũng có hai chiều ($r = 2$).

$$X = \begin{bmatrix} 1.0 & 0.5 \\ 0.0 & 1.0 \end{bmatrix}, \qquad Y = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}. \tag{7}$$

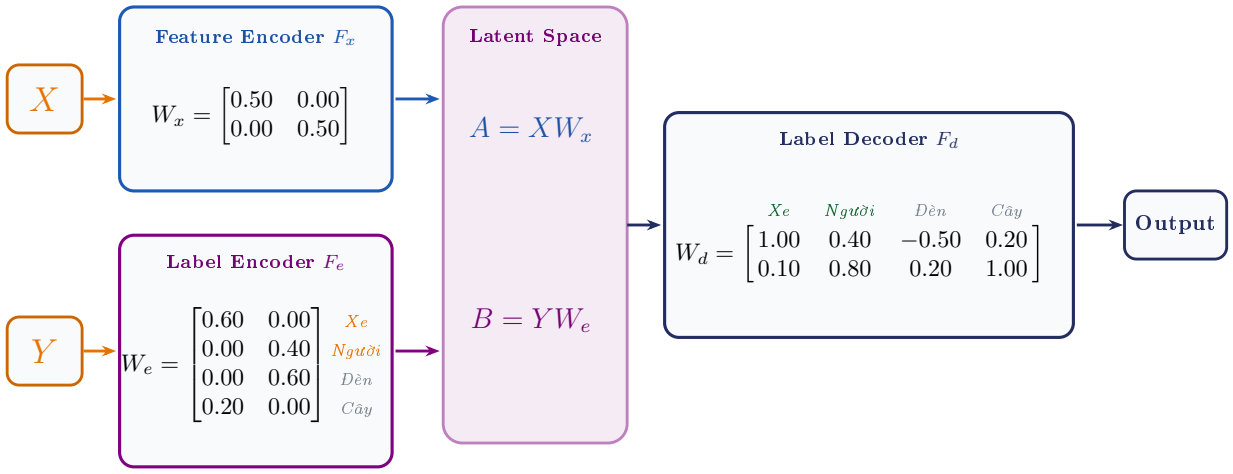
Dựa trên nhãn Y, chúng ta có thể đoán nội dung trực quan của hai mẫu ảnh:

Mẫu 1 Ảnh đường phố ban ngày có Xe và Người.

Mẫu 2 Ảnh công viên có Người và Cây.

2. Khởi tạo trọng số các mạng thành phần

Để theo dõi minh bạch từng số hạng, ta xét ba mạng nơ-ron tuyến tính (chưa kích hoạt phi tuyến và không có bias). Các ma trận trọng số W_x, W_e, W_d được khởi tạo như sau:



3. Bước 1: Tính Forward qua Feature Encoder (F_x)

Feature Encoder tính $A = XW_x$. Mỗi phần tử của A là tích vô hướng giữa một hàng của X và một cột của W_x:

$$\underbrace{\begin{bmatrix} 1.0 & 0.5 \\ 0.0 & 1.0 \end{bmatrix}}_{X} \times \underbrace{\begin{bmatrix} 0.50 & 0.00 \\ 0.00 & 0.50 \end{bmatrix}}_{W_x} = \underbrace{\begin{bmatrix} 0.50 & 0.25 \\ 0.00 & 0.50 \end{bmatrix}}_{A}$$

Mẫu 1

$$a_{11} = 1.0 \times 0.50 + 0.5 \times 0.00 = 0.50,$$
$$a_{12} = 1.0 \times 0.00 + 0.5 \times 0.50 = 0.25.$$

Mẫu 2

$$a_{21} = 0.0 \times 0.50 + 1.0 \times 0.00 = 0.00,$$
$$a_{22} = 0.0 \times 0.00 + 1.0 \times 0.50 = 0.50.$$

4. Bước 2: Tính Forward qua Label Encoder (F_e)

Label Encoder tính $B = YW_e$. Vì Y chứa các giá trị 0 và 1, mỗi mẫu cộng các hàng của W_e tương ứng với nhãn có mặt. Mẫu 1 chọn Xe và Người, còn mẫu 2 chọn Người và Cây:

$$\underbrace{\begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}}_{Y} \times \underbrace{\begin{bmatrix} 0.60 & 0.00 \\ 0.00 & 0.40 \\ 0.00 & 0.60 \\ 0.20 & 0.00 \end{bmatrix}}_{W_e} = \underbrace{\begin{bmatrix} 0.60 & 0.40 \\ 0.20 & 0.40 \end{bmatrix}}_{B}$$

Mẫu 1

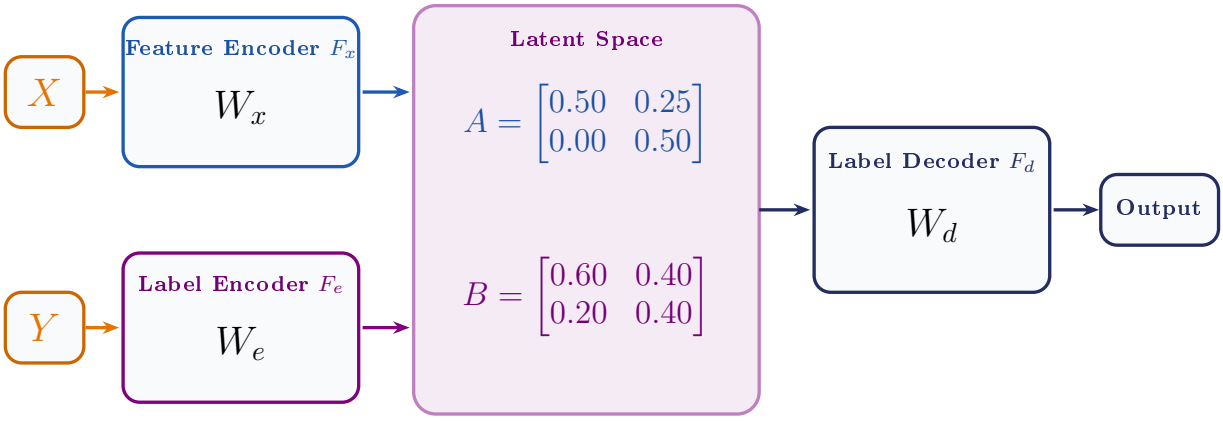
$$b_1^\top = [0.60,\ 0.00] + [0.00,\ 0.40] = [0.60,\ 0.40].$$

Mẫu 2

$$b_2^\top = [0.00,\ 0.40] + [0.20,\ 0.00] = [0.20,\ 0.40].$$

Trong ví dụ này, do cả hai mẫu đều mang nhãn Người, vector biểu diễn của chúng đều được tích lũy thêm vector trọng số $[0.00,\ 0.40]$. Đây chính là cơ chế giúp thông tin về một nhãn chung được phản ánh đồng thời vào không gian tiềm ẩn của cả hai mẫu.

Sau hai bước mã hóa, ta có hai biểu diễn A và B trong không gian tiềm ẩn:



5. Bước 3: Tính Forward qua Label Decoder (F_d)

Trong pha huấn luyện, Label Decoder nhận B và tính $S^{\text{train}} = BW_d$. Bốn cột của W_d lần lượt tạo điểm cho Xe, Người, Đèn và Cây:

$$\underbrace{\begin{bmatrix} 0.60 & 0.40 \\ 0.20 & 0.40 \end{bmatrix}}_{B} \times \underbrace{\begin{bmatrix} 1.00 & 0.40 & -0.50 & 0.20 \\ 0.10 & 0.80 & 0.20 & 1.00 \end{bmatrix}}_{W_d} = \underbrace{\begin{bmatrix} 0.64 & 0.56 & -0.22 & 0.52 \\ 0.24 & 0.40 & -0.02 & 0.44 \end{bmatrix}}_{S^{\text{train}}}$$

Mẫu 1

$$\begin{aligned} \text{Xe:} \quad & s_{11} = 0.60 \times 1.00 + 0.40 \times 0.10 = 0.64, \\ \text{Người:} \quad & s_{12} = 0.60 \times 0.40 + 0.40 \times 0.80 = 0.56, \\ \text{Đèn:} \quad & s_{13} = 0.60 \times (-0.50) + 0.40 \times 0.20 = -0.22, \\ \text{Cây:} \quad & s_{14} = 0.60 \times 0.20 + 0.40 \times 1.00 = 0.52. \end{aligned}$$

Mẫu 2

$$\begin{aligned} \text{Xe:} \quad & s_{21} = 0.20 \times 1.00 + 0.40 \times 0.10 = 0.24, \\ \text{Người:} \quad & s_{22} = 0.20 \times 0.40 + 0.40 \times 0.80 = 0.40, \\ \text{Đèn:} \quad & s_{23} = 0.20 \times (-0.50) + 0.40 \times 0.20 = -0.02, \\ \text{Cây:} \quad & s_{24} = 0.20 \times 0.20 + 0.40 \times 1.00 = 0.44. \end{aligned}$$

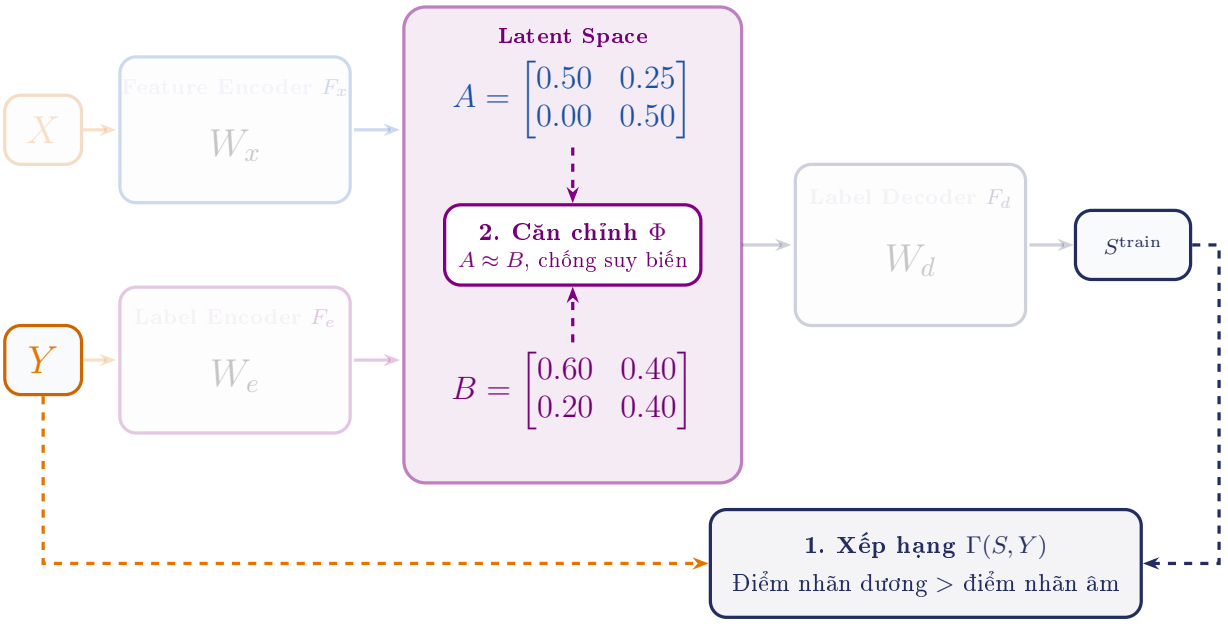
Các giá trị này là điểm số, chưa phải xác suất hay nhãn nhị phân. Chúng được dùng để tính mất mát xếp hạng ở mục tiếp theo.

Đến đây, chúng ta đã có đầy đủ ba thành phần:

1. Biểu diễn đặc trưng $A \in \mathbb{R}^{2\times 2}$.

2. Biểu diễn nhãn $B \in \mathbb{R}^{2\times 2}$.
3. Điểm số giải mã $S^{\text{train}} \in \mathbb{R}^{2\times 4}$.

Trong hai mục tiếp theo, chúng ta sẽ xem xét hai bài toán tối ưu chính: trước tiên là tối ưu bộ giải mã để điểm số S^{train} xếp hạng chính xác các nhãn dương cao hơn nhãn âm (Mục V.), sau đó là ép biểu diễn đặc trưng A khớp với biểu diễn nhãn B mà không bị suy biến không gian (Mục VI.):



V. Hàm mất mát xếp hạng nhãn $\Gamma(S, Y)$

Ở mẫu 1, ảnh có Xe và Người, không có Đèn và Cây. Decoder vừa cho các điểm số sau:

	Xe	Người	Đèn	Cây
Nhãn thật	1	1	0	0
Điểm số	0.64	0.56	-0.22	0.52
Nhóm nhãn	Dương	Dương	Âm	Âm

Ta muốn Xe và Người có điểm cao hơn Đèn và Cây. Bảng trên đã đúng thứ tự này, nhưng Người (0.56) chỉ cao hơn Cây (0.52) một chút. Cần một hàm mất mát phản ánh được mức độ phân tách giữa hai nhóm.

Ý tưởng của C2AE là so sánh từng cặp gồm một nhãn có mặt và một nhãn vắng mặt. Với mẫu 1, bốn cặp đó là Xe–Đèn, Xe–Cây, Người–Đèn và Người–Cây. Trong mỗi cặp, ta muốn điểm nhãn dương vượt lên trên điểm nhãn âm.

Để thực hiện điều đó, ta lấy điểm nhãn âm trừ điểm nhãn dương, thu được bảng sau:

Nhãn dương	**Nhãn âm**	**Hiệu điểm** d
Xe	Đèn	$-0.22 - 0.64 = -0.86$
Xe	Cây	$0.52 - 0.64 = -0.12$
Người	Đèn	$-0.22 - 0.56 = -0.78$
Người	Cây	$0.52 - 0.56 = -0.04$

- $d < 0$: nhãn dương cao điểm hơn nhãn âm (d càng âm thì khoảng cách chênh lệch điểm số càng lớn).
- $d = 0$: nhãn dương và nhãn âm có điểm bằng nhau.
- $d > 0$: nhãn âm cao điểm hơn nhãn dương, thứ tự xếp hạng bị đảo ngược.

Để biến hiệu này thành mức phạt dương, C2AE dùng hàm exponential [1]:

Nhãn dương	**Nhãn âm**	**Hiệu** d	**Mức phạt** $\exp(d)$
Xe	Đèn	-0.86	$e^{-0.86} \approx 0.4232$
Xe	Cây	-0.12	$e^{-0.12} \approx 0.8869$
Người	Đèn	-0.78	$e^{-0.78} \approx 0.4584$
Người	Cây	-0.04	$e^{-0.04} \approx 0.9608$

Lấy trung bình bốn mức phạt để được mất mát của mẫu 1:

$$E_1 \approx \frac{0.4232 + 0.8869 + 0.4584 + 0.9608}{4} \approx 0.6823. \tag{8}$$

Thực hiện tương tự cho mẫu 2, với nhãn dương là Người, Cây và nhãn âm là Xe, Đèn:

Nhãn dương	**Nhãn âm**	**Hiệu** d	**Mức phạt** $\exp(d)$
Người	Xe	$0.24 - 0.40 = -0.16$	$e^{-0.16} \approx 0.8521$
Người	Đèn	$-0.02 - 0.40 = -0.42$	$e^{-0.42} \approx 0.6570$
Cây	Xe	$0.24 - 0.44 = -0.20$	$e^{-0.20} \approx 0.8187$
Cây	Đèn	$-0.02 - 0.44 = -0.46$	$e^{-0.46} \approx 0.6313$

Mất mát của mẫu 2 là

$$E_2 \approx \frac{0.8521 + 0.6570 + 0.8187 + 0.6313}{4} \approx 0.7398. \tag{9}$$

Cộng hai mẫu để được mất mát của batch:

$$\Gamma = E_1 + E_2 \approx 0.6823 + 0.7398 = 1.4221. \tag{10}$$

Viết gọn quy trình này bằng ký hiệu, gọi P_i là tập chỉ số nhãn dương và Q_i là tập chỉ số nhãn âm của mẫu i. Điểm nhãn âm là s_{iq}, điểm nhãn dương là s_{ip}:

$$E_i = \frac{1}{|P_i||Q_i|} \sum_{p \in P_i} \sum_{q \in Q_i} \exp(s_{iq} - s_{ip}), \qquad \Gamma(S^{\text{train}}, Y) = \sum_{i=1}^{N} E_i. \tag{11}$$

Chỉ tính các mẫu có cả nhãn dương và nhãn âm để tạo được cặp so sánh.

Gradient của Γ truyền qua decoder để cập nhật F_d và F_e. Nhánh ảnh F_x học qua mục tiêu căn chỉnh Φ, được trình bày ở phần tiếp theo.

VI. Căn chỉnh không gian tiềm ẩn $\Phi(A, B)$

Decoder học từ biểu diễn nhãn B, nhưng khi suy luận lại nhận biểu diễn ảnh A. Vì vậy, C2AE vừa kéo A gần B, vừa ràng buộc cấu trúc của hai biểu diễn để tránh nghiệm suy biến.

1. Kéo hai biểu diễn lại gần nhau

Thành phần căn chỉnh cộng bình phương sai lệch ở từng vị trí:

$$\mathcal{L}_{\text{align}} = \|A - B\|_{\text{F}}^2 = \sum_{i=1}^{N} \sum_{k=1}^{r} (a_{ik} - b_{ik})^2. \tag{12}$$

Với hai ma trận đã tính, hàng xanh ứng với mẫu 1 và hàng tím ứng với mẫu 2:

$$\underbrace{\begin{bmatrix} 0.50 & 0.25 \\ 0.00 & 0.50 \end{bmatrix}}_{A} - \underbrace{\begin{bmatrix} 0.60 & 0.40 \\ 0.20 & 0.40 \end{bmatrix}}_{B} = \underbrace{\begin{bmatrix} -0.10 & -0.15 \\ -0.20 & 0.10 \end{bmatrix}}_{A-B}. \tag{13}$$

Mẫu 1 $(-0.10)^2 + (-0.15)^2 = 0.0325.$

Mẫu 2 $(-0.20)^2 + (0.10)^2 = 0.0500.$

$$\mathcal{L}_{\text{align}} = 0.0325 + 0.0500 = 0.0825. \tag{14}$$

Chỉ căn chỉnh là chưa đủ. Nếu $A = B = 0$, mất mát trên bằng 0 nhưng mọi mẫu đều có cùng biểu diễn, khiến decoder không phân biệt được chúng. Cần thêm một mục tiêu chống suy biến.

2. Giữ các chiều biểu diễn không suy biến

C2AE dùng các ma trận Gram $A^\top A$ và $B^\top B$. Mỗi phần tử là tích vô hướng giữa hai cột biểu diễn trên batch:

- Ô trên đường chéo là bình phương chuẩn độ dài của một cột, nên việc ép ô này về gần 1 sẽ chống nghiệm tầm thường (tránh các cột bị suy biến về 0).

- Ô ngoài đường chéo đo tích vô hướng giữa hai cột khác nhau, do đó việc đưa ô này về gần 0 sẽ định hướng các vector cột trở nên trực giao.

Ta phạt độ lệch của hai ma trận Gram so với ma trận đơn vị:

$$G_A = \left\|A^\top A - I_r\right\|_{\mathrm{F}}^2, \qquad G_B = \left\|B^\top B - I_r\right\|_{\mathrm{F}}^2. \tag{15}$$

Để tính mỗi giá trị, trước hết nhân ma trận chuyển vị với ma trận gốc, sau đó trừ 1 ở đường chéo và cộng bình phương mọi ô. Hai ô ngoài đường chéo bằng nhau nên được gộp thành hệ số 2:

Nhánh đặc trưng A

$$A^\top A = \begin{bmatrix} 0.50 & 0.00 \\ 0.25 & 0.50 \end{bmatrix} \begin{bmatrix} 0.50 & 0.25 \\ 0.00 & 0.50 \end{bmatrix} = \begin{bmatrix} 0.2500 & 0.1250 \\ 0.1250 & 0.3125 \end{bmatrix}$$

$$g_{11} = 0.50^2 + 0.00^2, \quad g_{12} = 0.50 \times 0.25 + 0.00 \times 0.50, \quad g_{22} = 0.25^2 + 0.50^2.$$

$$G_A = (0.2500 - 1)^2 + 2 \times 0.1250^2 + (0.3125 - 1)^2 = 1.0664\ldots$$

Nhánh nhãn B

$$B^\top B = \begin{bmatrix} 0.60 & 0.20 \\ 0.40 & 0.40 \end{bmatrix} \begin{bmatrix} 0.60 & 0.40 \\ 0.20 & 0.40 \end{bmatrix} = \begin{bmatrix} 0.40 & 0.32 \\ 0.32 & 0.32 \end{bmatrix}$$

$$g_{11} = 0.60^2 + 0.20^2, \quad g_{12} = 0.60 \times 0.40 + 0.20 \times 0.40, \quad g_{22} = 0.40^2 + 0.40^2.$$

$$G_B = (0.40 - 1)^2 + 2 \times 0.32^2 + (0.32 - 1)^2 = 1.0272.$$

3. Ghép hai thành phần thành $\Phi(A, B)$

Mất mát không gian tiềm ẩn của C2AE [1] gồm căn chỉnh và phạt Gram:

$$\Phi(A, B) = \mathcal{L}_{\text{align}} + \lambda(G_A + G_B). \tag{16}$$

Trong ví dụ tính tay, chọn $\lambda = 0.1$. Thay các kết quả vừa tính vào:

$$\Phi = 0.0825 + 0.1(G_A + G_B) \approx 0.0825 + 0.1 \times 2.0936 \approx 0.2919. \tag{17}$$

Hệ số λ điều chỉnh trọng số của mức phạt Gram so với căn chỉnh. Đây là siêu tham số cần được tinh chỉnh trên tập validation.

4. Hàm mục tiêu chung và cách cập nhật mô hình

Mục tiêu chung kết hợp Φ với mất mát xếp hạng Γ:

$$\min_{\theta_x,\theta_e,\theta_d} \mathcal{J} = \Phi(A, B) + \alpha\Gamma(S^{\text{train}}, Y). \tag{18}$$

Chọn $\alpha = 0.5$ cho ví dụ, ta được

$$\mathcal{J} = \Phi + 0.5\Gamma \approx 1.0029. \tag{19}$$

Hai loss cập nhật các mạng theo những đường khác nhau:

Mạng	Nhận gradient từ	Vai trò
F_x	Φ	Đưa biểu diễn ảnh gần biểu diễn nhãn và giảm phạt Gram.
F_e	Φ và Γ	Học biểu diễn nhãn vừa khớp nhánh ảnh, vừa phục vụ xếp hạng.
F_d	Γ	Học cách giải mã biểu diễn nhãn thành điểm số.

VII. Suy luận và chọn ngưỡng dự đoán

1. Dùng nhánh ảnh thay cho nhánh nhãn

Khi suy luận, mô hình chỉ nhận đặc trưng đầu vào. Luồng dữ liệu đi qua F_x rồi đến F_d, trong khi nhánh F_e hoàn toàn không tham gia vào quá trình này:

$$S^{\text{infer}} = F_d(F_x(X)). \tag{20}$$

Trong ví dụ tuyến tính, phép tính này là $S^{\text{infer}} = AW_d$. Ở bước này, ta tiếp tục áp dụng các ma trận trọng số khởi tạo ban đầu để đối chiếu luồng truyền dữ liệu giữa hai pha:

$$\underbrace{\begin{bmatrix} 0.50 & 0.25 \\ 0.00 & 0.50 \end{bmatrix}}_{A} \times \underbrace{\begin{bmatrix} 1.00 & 0.40 & -0.50 & 0.20 \\ 0.10 & 0.80 & 0.20 & 1.00 \end{bmatrix}}_{W_d} = \underbrace{\begin{bmatrix} 0.525 & 0.400 & -0.200 & 0.350 \\ 0.050 & 0.400 & 0.100 & 0.500 \end{bmatrix}}_{S^{\text{infer}}}$$

Mẫu 1

$$\text{Điểm Xe} = 0.50 \times 1.00 + 0.25 \times 0.10 = 0.525.$$

Mẫu 2

$$\text{Điểm Xe} = 0.00 \times 1.00 + 0.50 \times 0.10 = 0.050.$$

Hai phép tính dưới hình minh họa cách tính điểm cho nhãn Xe, còn các điểm số còn lại được tính tương tự bằng cách nhân hàng của A với từng cột tương ứng của W_d.

Đặt các điểm số cạnh nhau để so sánh:

Mẫu	Luồng	Xe	Người	Đèn	Cây
1	Huấn luyện, từ B	0.640	0.560	-0.220	0.520
1	Suy luận, từ A	0.525	0.400	-0.200	0.350
2	Huấn luyện, từ B	0.240	0.400	-0.020	0.440
2	Suy luận, từ A	0.050	0.400	0.100	0.500

Điểm số khác nhau vì $A \neq B$. Trong hai mẫu này, các nhãn dương vẫn xếp trên nhãn âm ở cả hai luồng.

2. Đổi điểm số thành nhãn

Điểm đầu ra là số thực, không phải xác suất. Ta chọn một ngưỡng t và gán nhãn theo quy tắc

$$\widehat{y}_{ij} = \begin{cases} 1, & s_{ij}^{\text{infer}} \geq t, \\ 0, & s_{ij}^{\text{infer}} < t. \end{cases} \tag{21}$$

Với ngưỡng minh họa $t = 0.38$, so sánh từng cột với ngưỡng để tạo vector dự đoán:

Mẫu 1

	Xe	Người	Đèn	Cây
s_i^{infer}	0.525	0.400	-0.200	0.350
So với 0.38	$\geq$	$\geq$	$<$	$<$
$\widehat{y}_i$	1	1	0	0

Mẫu 2

	Xe	Người	Đèn	Cây
s_i^{infer}	0.050	0.400	0.100	0.500
So với 0.38	$<$	$\geq$	$<$	$\geq$
$\widehat{y}_i$	0	1	0	1

Hai vector thu được lần lượt là $[1, 1, 0, 0]$ và $[0, 1, 0, 1]$, khớp với nhãn thật của hai mẫu.

3. Chọn ngưỡng trên tập validation

Trong thực hành, chọn ngưỡng bằng dữ liệu validation rồi giữ cố định khi đánh giá test:

1. Chạy suy luận trên tập validation để thu điểm số và chuẩn bị các ngưỡng ứng viên.
2. Với mỗi ngưỡng, tạo nhãn dự đoán và tính Micro-F1, sau đó chọn ngưỡng đem lại điểm số cao nhất.
3. Giữ nguyên mô hình và ngưỡng đã chọn, rồi đánh giá một lần trên tập test.

Trong thực tế triển khai, giá trị ngưỡng tối ưu t^* bắt buộc phải được tìm kiếm trên tập Validation nhằm đảm bảo tính khách quan trước khi đánh giá trên tập Test.

VIII. Các độ đo chuyên dụng cho bài toán đa nhãn

Một mẫu đa nhãn có thể đúng ở nhãn này nhưng sai ở nhãn khác. Vì vậy, ta cần phân biệt tỷ lệ ô nhãn sai, tỷ lệ mẫu đúng hoàn toàn và khả năng nhận ra các nhãn dương.

Ta dùng chung ví dụ gồm bốn mẫu và bốn nhãn dưới đây để tính từng độ đo. Ô cam đánh dấu nhãn bị bỏ sót (FN, từ 1 thành 0), còn ô tím đánh dấu nhãn bị dự đoán thừa (FP, từ 0 thành 1):

Bảng 2: Nhãn thật và dự đoán của bốn mẫu dùng để tính các độ đo.

Mẫu	Dữ liệu	Xe	Người	Đèn	Cây
1	Nhãn thật	1	1	0	0
	Dự đoán	1	0 (FN)	0	0
2	Nhãn thật	0	1	0	1
	Dự đoán	0	1	0	1
3	Nhãn thật	1	0	1	0
	Dự đoán	1	0	0 (FN)	0
4	Nhãn thật	0	0	0	1
	Dự đoán	0	0	1 (FP)	1

1. Hamming Loss: Sai bao nhiêu ô nhãn?

Hamming Loss là số ô dự đoán sai chia cho tổng số ô nhãn:

$$\mathcal{H}(Y, \widehat{Y}) = \frac{1}{NL} \sum_{i=1}^{N} \sum_{j=1}^{L} \mathbb{I}[y_{ij} \neq \widehat{y}_{ij}]. \tag{22}$$

Hàm chỉ báo $\mathbb{I}[\cdot]$ bằng 1 khi điều kiện đúng và 0 khi điều kiện sai. Hamming Loss càng nhỏ càng tốt.

Trong bảng, mẫu 1 sai nhãn Người, mẫu 3 sai nhãn Đèn và mẫu 4 cũng sai nhãn Đèn. Có ba ô sai trên $4 \times 4 = 16$ ô:

$$\mathcal{H} = \frac{3}{16} = \boxed{0.1875}. \tag{23}$$

Như vậy, tỷ lệ ô nhãn đúng là $1 - \mathcal{H} = 81.25\%$.

2. Subset Accuracy: Bao nhiêu mẫu đúng hoàn toàn?

Subset Accuracy chỉ tính một mẫu là đúng khi toàn bộ vector dự đoán khớp với nhãn thật:

$$\text{SubsetAcc}(Y, \widehat{Y}) = \frac{1}{N} \sum_{i=1}^{N} \mathbb{I}[y_i = \widehat{y}_i]. \tag{24}$$

Trong ví dụ, chỉ mẫu 2 đúng cả bốn nhãn. Ba mẫu còn lại đều sai một nhãn nên không được tính là đúng:

$$\text{SubsetAcc} = \frac{1}{4} = \boxed{25\%}. \tag{25}$$

Cùng một bảng dự đoán, tỷ lệ ô đúng đạt 81.25% nhưng tỷ lệ mẫu đúng hoàn toàn chỉ đạt 25%. Hai độ đo trả lời hai câu hỏi khác nhau.

3. Precision, Recall và F1 của từng nhãn

Khi phần lớn ô nhãn là 0, tỷ lệ ô đúng dễ che khuất việc bỏ sót nhãn dương. Chẳng hạn, nếu 95% ô là 0, dự đoán mọi ô bằng 0 đạt tỷ lệ ô đúng 95% nhưng không tìm được nhãn dương nào. Precision, Recall và F1 giúp đánh giá phần dự đoán nhãn dương.

Với nhãn j, ta đếm ba trường hợp:

- TP_j: có nhãn và dự đoán có.
- FP_j: không có nhãn nhưng dự đoán có.
- FN_j: có nhãn nhưng dự đoán không có.

Precision đo tỷ lệ chính xác trong các lần mô hình dự đoán có nhãn, trong khi Recall đo tỷ lệ phát hiện thành công những nhãn thực sự xuất hiện:

$$\text{Precision}_j = \frac{TP_j}{TP_j + FP_j}, \qquad \text{Recall}_j = \frac{TP_j}{TP_j + FN_j}. \tag{26}$$

Xét nhãn Người: Mẫu 2 được nhận diện đúng còn mẫu 1 bị bỏ sót, đồng thời không có dự đoán thừa nào. Do đó $TP = 1$, $FP = 0$, $FN = 1$:

$$\text{Precision}_{\text{Người}} = \frac{1}{1+0} = \boxed{1}, \qquad \text{Recall}_{\text{Người}} = \frac{1}{1+1} = \boxed{0.5}. \tag{27}$$

F1 là trung bình điều hòa của Precision và Recall, có thể tính trực tiếp từ ba số đếm:

$$F1_j = \frac{2TP_j}{2TP_j + FP_j + FN_j}. \tag{28}$$

Vẫn với nhãn Người, ta được

$$F1_{\text{Người}} = \frac{2 \times 1}{2 \times 1 + 0 + 1} = \frac{2}{3} \approx \boxed{0.6667}. \tag{29}$$

Làm tương tự cho bốn nhãn, thu được bảng số đếm và điểm F1:

Nhãn	TP	FP	FN	$F1$
Xe	2	0	0	1
Người	1	0	1	2/3
Đèn	0	1	1	0
Cây	2	0	0	1
Tổng	5	1	2	

4. Micro-F1: Gộp số đếm rồi tính F1

Micro-F1 cộng TP, FP, FN của mọi nhãn trước khi tính F1:

$$F1_{\text{micro}} = \frac{2\sum_{j=1}^{L} TP_j}{2\sum_{j=1}^{L} TP_j + \sum_{j=1}^{L} FP_j + \sum_{j=1}^{L} FN_j}. \tag{30}$$

Dòng tổng trong bảng cho $TP = 5$, $FP = 1$, $FN = 2$. Thay trực tiếp vào công thức:

$$F1_{\text{micro}} = \frac{2 \times 5}{2 \times 5 + 1 + 2} = \frac{10}{13} \approx \boxed{76.92\%}. \tag{31}$$

Độ đo này đánh giá chung các dự đoán nhãn dương trên toàn tập dữ liệu. Nhãn xuất hiện nhiều đóng góp nhiều lần vào các số đếm.

5. Macro-F1: Tính F1 rồi lấy trung bình các nhãn

Macro-F1 tính F1 riêng cho từng nhãn rồi lấy trung bình, mỗi nhãn có trọng số bằng nhau:

$$F1_{\text{macro}} = \frac{1}{L}\sum_{j=1}^{L} F1_j. \tag{32}$$

Lấy bốn giá trị F1 trong bảng, ta được

$$F1_{\text{macro}} = \frac{1 + \frac{2}{3} + 0 + 1}{4} = \frac{2}{3} \approx \boxed{66.67\%}. \tag{33}$$

Nhãn Đèn có $F1 = 0$ nhưng vẫn chiếm một phần tư trọng số khi tính Macro-F1. Vì vậy, điểm thấp của một nhãn không bị lấn át bởi số lần dự đoán đúng ở các nhãn khác.

Hamming Loss và Subset Accuracy cho biết mức độ đúng theo ô và theo mẫu. Micro-F1 và Macro-F1 bổ sung hai góc nhìn về khả năng nhận ra nhãn dương, lần lượt theo tổng số đếm và theo từng nhãn.

IX. Triển khai thực tế mô hình C2AE với PyTorch

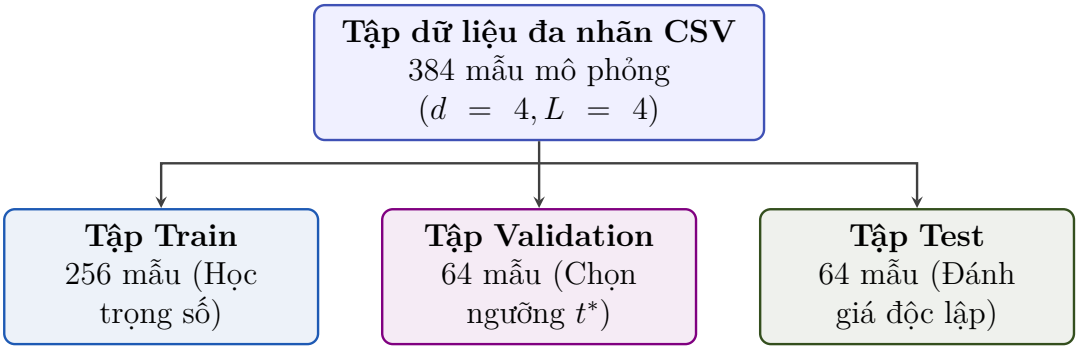
Để đưa toàn bộ lý thuyết và các công thức tính tay ở trên vào thực tế, trong mục này chúng ta sẽ cùng xây dựng một pipeline hoàn chỉnh bằng **PyTorch**, từ định nghĩa kiến trúc mạng, xây dựng hàm mất mát tùy biến (*Custom Loss Functions*), huấn luyện với thuật toán Adam, quét tìm ngưỡng tối ưu trên tập Validation, cho đến đánh giá toàn diện trên tập Test.

1. Thiết lập bài toán và cấu trúc dữ liệu mô phỏng

Ta sử dụng bộ dữ liệu mô phỏng gồm 384 mẫu. Mỗi mẫu có $d = 4$ đặc trưng số học và một vector nhãn trên $L = 4$ lớp Xe, Người, Đèn, Cây. Dữ liệu được chia thành ba tập:

- **Tập Huấn luyện (Train)**: 256 mẫu dùng để học trọng số mô hình.

- **Tập Xác thực (Validation)**: 64 mẫu dùng để theo dõi quá trình học và cân chỉnh ngưỡng tối ưu t^*.
- **Tập Kiểm thử (Test)**: 64 mẫu độc lập chỉ dùng một lần duy nhất để đánh giá hiệu năng cuối cùng.



Hình 23: Quy trình phân chia dữ liệu và vai trò nghiêm ngặt của từng phân tập.

2. Bước 1: Chuẩn hóa đặc trưng đầu vào

Ta chuẩn hóa từng đặc trưng bằng cách trừ trung bình và chia cho độ lệch chuẩn (*Z-score Normalization*), giúp các đặc trưng có thang đo tương đương khi đưa vào mạng. Để tránh hiện tượng rò rỉ dữ liệu (*data leakage*), các tham số chuẩn hóa (μ, σ) được tính toán độc lập trên tập Train, sau đó áp dụng cố định cho tập Validation và Test:

Python: Chuẩn hóa dữ liệu

```python
import torch
import numpy as np

def standardize_data(x_train, x_val, x_test):
    # Compute feature statistics from the training set only.
    mean = x_train.mean(dim=0, keepdim=True)
    std = x_train.std(dim=0, keepdim=True)
    std[std == 0] = 1.0  # Avoid division by zero for constant features.

    x_train_norm = (x_train - mean) / std
    x_val_norm = (x_val - mean) / std
    x_test_norm = (x_test - mean) / std
    return x_train_norm, x_val_norm, x_test_norm, mean, std
```

3. Bước 2: Xây dựng kiến trúc mô hình C2AE

Lớp `C2AE` kế thừa `nn.Module` của PyTorch, bao gồm ba mạng thành phần:

- `fx`: Feature Encoder ($\mathbb{R}^d \to \mathbb{R}^{\text{hidden}} \to \mathbb{R}^r$).
- `fe`: Label Encoder ($\mathbb{R}^L \to \mathbb{R}^{\text{hidden}} \to \mathbb{R}^r$).
- `fd`: Label Decoder ($\mathbb{R}^r \to \mathbb{R}^{\text{hidden}} \to \mathbb{R}^L$).

Python: Lớp mô hình C2AE

```python
import torch.nn as nn

class C2AE(nn.Module):
    def __init__(self, feature_dim=4, label_dim=4, latent_dim=2, hidden=16):
        super().__init__()
        # 1. Feature encoder: F_x
        self.fx = nn.Sequential(
            nn.Linear(feature_dim, hidden),
            nn.Tanh(),
            nn.Linear(hidden, latent_dim)
        )
        # 2. Label encoder: F_e
        self.fe = nn.Sequential(
            nn.Linear(label_dim, hidden),
            nn.Tanh(),
            nn.Linear(hidden, latent_dim)
        )
        # 3. Label decoder: F_d
        self.fd = nn.Sequential(
            nn.Linear(latent_dim, hidden),
            nn.Tanh(),
            nn.Linear(hidden, label_dim)
        )

    def forward(self, x):
        """Predict label scores from input features during inference."""
        a = self.fx(x)          # Encode input features.
        scores = self.fd(a)     # Decode feature embeddings.
        return scores

    def training_outputs(self, x, y):
        """Return both embeddings and label-branch scores for training."""
        a = self.fx(x)          # Feature embeddings (N x r).
        b = self.fe(y)          # Label embeddings (N x r).
        s_train = self.fd(b)    # Decode label embeddings (N x L).
        return a, b, s_train
```

4. Bước 3: Cài đặt hai hàm mất mát tùy biến

Ta hiện thực hóa chính xác công thức toán học của Γ (Mục V.) và Φ (Mục VI.):

Python: Hàm mất mát Latent và Ranking

```python
def latent_loss(a, b, lam=0.1):
    """Compute alignment and Gram penalties for Phi(A, B)."""
    # 1. Alignment loss
```

```
    align_loss = torch.sum((a - b) ** 2)

    # 2. Gram matrices and orthonormality penalty
    r = a.shape[1]
    I_r = torch.eye(r, device=a.device, dtype=a.dtype)

    gram_a = torch.mm(a.t(), a)
    gram_b = torch.mm(b.t(), b)

    gram_penalty = torch.sum((gram_a - I_r) ** 2) + torch.sum((gram_b -
        I_r) ** 2)

    total_phi = align_loss + lam * gram_penalty
    return total_phi, align_loss, gram_penalty

def ranking_loss(scores, y):
    """Sum exponential pairwise ranking losses over the batch."""
    N, L = scores.shape
    total_gamma = 0.0

    for i in range(N):
        pos_idx = (y[i] == 1).nonzero(as_tuple=True)[0]
        neg_idx = (y[i] == 0).nonzero(as_tuple=True)[0]

        # Skip samples without both positive and negative labels.
        if len(pos_idx) == 0 or len(neg_idx) == 0:
            continue

        s_pos = scores[i, pos_idx]  # Positive label scores (|P_i|).
        s_neg = scores[i, neg_idx]  # Negative label scores (|Q_i|).

        # Pairwise score differences: shape (|Q_i|, |P_i|).
        diff = s_neg.unsqueeze(1) - s_pos.unsqueeze(0)
        pair_penalties = torch.exp(diff)

        e_i = torch.mean(pair_penalties)
        total_gamma = total_gamma + e_i

    return total_gamma
```

5. Bước 4: Huấn luyện và Cân chỉnh ngưỡng tối ưu

Vòng lặp huấn luyện tối ưu hàm mục tiêu $\mathcal{J} = \Phi + \alpha\Gamma$ bằng thuật toán Adam. Ở epoch đầu và sau mỗi 10 epoch, mô hình quét 41 giá trị ngưỡng trên tập validation để chọn cặp checkpoint và ngưỡng có Micro-F1 cao nhất trong các lần thử:

Python: Training Loop và Quét ngưỡng

```
from sklearn.metrics import f1_score
import copy
```

```python

def evaluate_threshold(val_scores, y_val, t):
    pred = (val_scores >= t).astype(int)
    return f1_score(y_val, pred, average='micro', zero_division=0)

def fit_c2ae(model, optimizer, x_train, y_train, x_val, y_val,
             epochs=120, batch_size=64, alpha=0.5, lam=0.1):
    best_val_f1 = -1.0
    best_threshold = 0.0
    best_weights = None

    for epoch in range(1, epochs + 1):
        model.train()
        permutation = torch.randperm(x_train.shape[0])

        for i in range(0, x_train.shape[0], batch_size):
            indices = permutation[i:i + batch_size]
            batch_x, batch_y = x_train[indices], y_train[indices]

            optimizer.zero_grad()
            a, b, s_train = model.training_outputs(batch_x, batch_y)

            phi, _, _ = latent_loss(a, b, lam)
            gamma = ranking_loss(s_train, batch_y)
            joint_loss = phi + alpha * gamma

            joint_loss.backward()
            optimizer.step()

        # Tune the threshold at epoch 1 and every 10 epochs.
        if epoch == 1 or epoch % 10 == 0:
            model.eval()
            with torch.no_grad():
                val_scores = model(x_val).cpu().numpy()
            y_val_np = y_val.cpu().numpy()

            # Evaluate 41 candidate thresholds.
            candidates = np.linspace(val_scores.min() - 1e-3,
                                     val_scores.max() + 1e-3, 41)
            for t in candidates:
                f1_val = evaluate_threshold(val_scores, y_val_np, t)
                if f1_val > best_val_f1:
                    best_val_f1 = f1_val
                    best_threshold = t
                    best_weights = copy.deepcopy(model.state_dict())

    # Restore the checkpoint with the best validation Micro-F1.
    model.load_state_dict(best_weights)
    return model, best_threshold, best_val_f1
```

6. Bước 5: Đánh giá độc lập trên tập Test

Sau huấn luyện, ta giữ cố định checkpoint và ngưỡng đã chọn trên validation. Trong lần chạy được báo cáo, ngưỡng tối ưu tìm được là $t^* = -0.3992$. Mô hình được đánh giá trên 64 mẫu test độc lập:

Bảng 3: Báo cáo hiệu năng thực nghiệm của mô hình C2AE trên tập Test (64 mẫu độc lập).

Phương pháp	Hamming Loss ↓	Subset Accuracy ↑	Micro-F1 ↑	Macro-F1 ↑
C2AE	0.2891	0.3594	0.7218	0.7232

Đọc bảng kết quả

- **Theo từng ô và từng mẫu**: Hamming Loss 0.2891 tương ứng 74 ô nhãn sai trên 256 ô. Subset Accuracy 0.3594 tương ứng 23/64 mẫu được dự đoán đúng toàn bộ nhãn.
- **Theo nhãn dương**: Micro-F1 đạt 0.7218 và Macro-F1 đạt 0.7232. Hai chỉ số này khá tương đồng. Để xác định cụ thể nhãn nào còn yếu, ta có thể phân tích thêm F1 của từng nhãn như cách tính ở Mục VIII..

X. Tổng kết

C2AE học biểu diễn đặc trưng và biểu diễn nhãn trong cùng một không gian tiềm ẩn. Cơ chế của mô hình có thể tóm tắt qua ba điểm:

- **Ba mạng với vai trò rõ ràng**: F_x mã hóa đặc trưng đầu vào thành A, F_e mã hóa nhãn thành B, còn F_d giải mã biểu diễn tiềm ẩn thành điểm số cho từng nhãn.
- **Hai mục tiêu huấn luyện**: Γ so sánh điểm của từng cặp nhãn dương–âm trên đầu ra $F_d(B)$. Φ kéo A gần B và dùng phạt Gram để hạn chế suy biến biểu diễn. Hai thành phần được kết hợp trong $\mathcal{J} = \Phi + \alpha\Gamma$.
- **Suy luận qua nhánh đặc trưng**: Khi dự đoán, mô hình dùng $F_d(F_x(X))$, rồi phân ngưỡng để tạo nhãn nhị phân. Nhánh mã hóa nhãn chỉ được dùng trong huấn luyện.

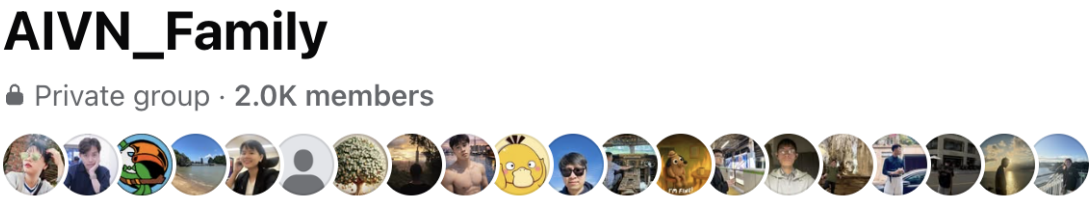
Ví dụ tính tay dùng các mạng tuyến tính để làm rõ từng phép nhân ma trận, cách tính loss và cách áp dụng ngưỡng. Phần PyTorch triển khai các mạng có kích hoạt phi tuyến, tối ưu trọng số trên tập train, chọn checkpoint và ngưỡng trên tập validation, rồi đánh giá trên tập test.

Khi đọc kết quả, Hamming Loss cho biết tỷ lệ ô nhãn sai, Subset Accuracy đo tỷ lệ mẫu đúng hoàn toàn, còn Micro-F1 và Macro-F1 lần lượt tổng hợp theo số đếm và theo từng nhãn. Kết hợp các độ đo này giúp đọc kết quả ở mức ô nhãn, mức mẫu và mức nhãn.

Tài liệu tham khảo

[1] C.-K. Yeh et al., "Learning deep latent space for multi-label classification", in *Proceedings of the Thirty-First AAAI Conference on Artificial Intelligence (AAAI-17)*, AAAI Press, 2017, pp. 2838–2844. [Online]. Available: https://ojs.aaai.org/index.php/AAAI/article/view/10769

[2] M.-L. Zhang et al., "A review on multi-label learning algorithms", *IEEE Transactions on Knowledge and Data Engineering*, vol. 26, no. 8, pp. 1819–1837, 2014. DOI: 10.1109/TKDE.2013.39 [Online]. Available: https://doi.org/10.1109/TKDE.2013.39



Cộng đồng học tập và nghiên cứu Trí tuệ Nhân tạo AIO.

Mã nguồn thực hành

Toàn bộ mã nguồn thực hành Google Colab và tệp dữ liệu mô phỏng đi kèm bài viết được lưu trữ và cập nhật công khai tại:

Truy cập Google Colab Notebook C2AE